

Solar Battery System

User Guide

Sigenergy 24 kWh Battery | 20 kW Solar | Globird Zero Hero Tariff

Automated daily energy optimisation — Solar forecasting, battery management, spa control, and revenue maximisation

System	Sigenergy 24 kWh 15% minimum reserve = 3.6 kWh usable floor
Solar	20 kW rooftop
Retailer	Globird Zero Hero (example tariff — adapt to your own)
Location	[Your location — update paths and timezone as needed]
Document date	June 2026
Status	Live system — self-calibrating daily from actual overnight outcomes

Items shown in [square brackets] throughout this document are placeholders. Replace them with your own Station ID, file paths, IP addresses, and API keys before use.

1. What This System Does

Your home generates solar power during the day and stores excess energy in a 24 kWh Sigenergy battery. The Globird Zero Hero tariff lets you maximise value from this storage with two special pricing windows:

Tariff window	Hours	Rate	What it means
Free grid charging	11am – 2pm	Free (zero cost)	Grid tops up the battery at no charge
Premium export	6pm – 9pm	\$0.15 / kWh	Sell stored energy to the grid for 15c per kWh
Base export	5pm – 11pm	\$0.05 / kWh	Standard feed-in rate applies in this window only
No feed-in	All other times	\$0.00 / kWh	No credit for energy exported outside 5–11pm

The challenge: you need to decide each evening how much energy to export at 6pm. Export too much and the battery runs out before morning — you pay grid rates overnight. Export too little and you leave money on the table. The decision depends on how much solar you generated, what the overnight discharge rate will be, and what tomorrow looks like.

This system solves that problem automatically. Every day it:

- Fetches tomorrow's solar forecast from Solcast
- Learns from last night's battery behaviour to sharpen its discharge model
- Decides whether to run the spa on solar power (saving you money vs grid heating)
- Calculates the optimal export quantity and sends you a recommendation by 4:30pm
- Tracks actual outcomes each morning and adjusts its targets over time

The system never acts autonomously on the export decision — it sends you a recommendation and you set the export mode yourself. All other actions (spa on/off) are fully automated.

2. Your Setup

Retailer Tariff

Retailer	Globird Zero Hero (example — adapt to your own tariff)
Free grid window	11:00am – 2:00pm (3 hours daily)
Premium export	6:00pm – 9:00pm @ \$0.15/kWh (10c feed-in + 5c bonus)
Standard export	5pm – 11pm @ \$0.05/kWh
Goal (5–11pm)	Avoid importing from grid — you lose the 5c/kWh no-import credit
Battery reserve	15% minimum (3.6 kWh) — never discharged below this

Hardware

Battery	Sigenergy 24 kWh Station ID: [Your Station ID — from Sigenergy app]
Usable capacity	20.4 kWh (24 kWh minus 15% = 3.6 kWh reserve)
Solar panels	20 kW rooftop
Spa controller	Gecko in.touch 2 IP: [Your Spa IP — check router or Gecko app]
Spa heater draw	~3.2 kW when heating (3.0 kW element + 0.2 kW circulation pump)

Solar Forecast

Provider	Solcast (solcast.com.au)
Resource ID	[Your Resource ID — from Solcast dashboard]
Panel tilt	15 degrees (example — enter your actual tilt)
Panel facing	North — 0 degrees (example — enter your actual azimuth)
Fetches at	3:00pm daily — provides tomorrow's forecast
Classes	HIGH (>25 kWh), MEDIUM (12–25 kWh), LOW (4–12 kWh), VERY LOW (<4 kWh)

Where Files Live

Scripts & data	C:\Solar\ (example path — use whatever suits your machine)
Scheduled tasks	C:\Solar\Scheduled\ (Claude scheduled task files)
Sigen API	https://app-aus.sigencloud.com (Chrome extension, automatic login)
Calibration	sigen_calibration.json (the system brain — never delete)

All file paths in this guide use C:\Solar\ as a generic example. On your machine, replace this with your actual path (e.g. C:\Users\YourName\Projects\Solar\).

3. Daily Timeline — What Happens and When

The system runs entirely on Windows Task Scheduler. Nothing needs to be open on your computer. Here is what happens each day:

~8:00am	Morning Learning Task runs. Connects to the Sigen API via Chrome, reads last night's battery data, updates the discharge model, and sends you a morning summary email showing how last night went and what changed in the calibration.
10:00am	Spa Decision (10am). Checks the solar forecast and battery level. If the battery has at least 9 kWh (37.5%) at 10am and the spa water needs heating, turns the spa on to run from solar power from 10am to 11am (battery only — free grid starts at 11am).
11:00am	Free grid window begins. Grid + solar charge the battery for free. The spa continues running but now draws from free grid power, not battery.
2:00pm	Free grid window ends. Battery is typically at 100%. Spa Afternoon Decision runs — uses the Solcast forecast to decide if solar is strong enough to extend the spa for one more hour (2–3pm). If the projected battery level at 4pm would still be above 95%, the spa keeps running.
3:00pm	Spa 3pm Decision runs. If the 2pm decision extended to 3pm, this script checks whether a further extension to 4pm is safe. If not approved, heat_off is called. Solar-SpaOff task also always runs heat_off at 4pm as a final safety net.
~3:00pm	Solcast Forecast Fetch. Downloads tomorrow's solar forecast from the Solcast API and saves it to solcast_forecast.json.
~4:30pm	Nightly Battery Recommendation. Reads the actual battery SOC at 4pm from the Sigen API, calculates the optimal export quantity for 6–9pm based on the discharge model and tomorrow's solar forecast, and emails you a recommendation.
6:00pm	You set the export mode on the Globird/Sigen app based on the recommendation. Round to the nearest whole number is fine — the system accounts for this.
6:00–9pm	Battery exports to grid at the rate you set. System records the actual export kWh for next morning's learning.
~8:00am	Next morning: the learning task runs again and compares what actually happened to what was projected.

The 5-minute email task (Solar-SendPendingEmail) runs continuously in the background. Both the morning and nightly tasks write an email to a queue file — the 5-minute task detects it and sends it, then deletes the queued file.

4. The Scripts — How Each One Works

4.1 fetch_solcast.py — Solar Forecast

`fetch_solcast.py`

Runs at 3:00pm daily via Task Scheduler (Solar-FetchSolcast)

Connects to the Solcast API and downloads tomorrow's solar generation forecast for your panels. The forecast is used by the 10am spa decision and the 4:30pm nightly recommendation.

What it produces

Field in solcast_forecast.json	What it means
solar_class	Tomorrow's day type: HIGH, MEDIUM, LOW, or VERY_LOW
daily_kwh	Estimated total generation tomorrow (kWh)
morning_kwh	Estimated generation 8am–11am
afternoon_kwh	Estimated generation 2pm–5pm
forecast_date	The date this forecast applies to (tomorrow)

If Solcast is unavailable at 3pm, the 4:30pm recommendation falls back to the Weatherzone classification and the last known Solcast reading. The spa decision scripts also have fallback logic.

4.2 spa_solar_decision.py — Should the Spa Heat at 10am?

`spa_solar_decision.py`

Runs at 10:00am daily via Task Scheduler (Solar-SpaDecision)

One hour before free grid starts, this script decides whether to turn the spa on. Running the spa at 10am costs battery power (3.2 kW for one hour = up to 3.2 kWh), but free grid kicks in at 11am and replenishes the battery. The question is: does the battery have enough of a cushion to safely run the spa for that one battery-only hour?

Decision logic

Check	Threshold	What happens if failed
Projected battery at 10am	Must be ≥ 9 kWh (37.5%)	NO_ACTION — spa stays off
Water temp vs target	Water must be at least 1°C below target (38.5°C)	NO_ACTION — not worth heating
Both checks pass	—	HEAT_ON — spa turned on to 38.5°C target

How battery level at 10am is estimated

The script cannot read the live battery at 10am — it runs before connecting to the Sigen API. Instead it projects forward from the 8am reading:

- The 8am SOC is read from morning_data.json (written by the 8am learning task — actual reading)
- Solar recharge model predicts how much the battery charges from 8am to 10am based on the solar forecast class
- Projected 10am SOC = 8am SOC + (2 hours x solar recharge rate for today's class)
- Note: the field is called soc_7am in the code but always holds the 8am reading — the learning task uses the 08:00 API record
- If morning_data.json is missing or from yesterday, it falls back to the rolling-average 7am SOC from the calibration file

Output

- Decision logged to spa_decision_log.json
- Water temperature and decision saved to spa_daily_temps.json (used by nightly report)
- spa_control.py called with heat_on if approved

4.3 spa_afternoon_decision.py — Extend the Spa to 3pm?

spa_afternoon_decision.py

Runs at 2:00pm daily via Task Scheduler (Solar-SpaAfternoon)

Free grid just ended and the battery is at 100%. The spa has been running since 10am or 11am. This script asks: can we keep the spa running for one more hour (2–3pm) without draining the battery so low that it affects tonight's export decision?

The physics calculation

The target is to have at least 95% SOC (22.8 kWh) at 4pm — leaving enough headroom for the nightly recommendation. The script projects forward using the Solcast 2–5pm afternoon forecast:

- solar_avg_kw = afternoon_kwh (Solcast 2–5pm forecast) divided by 3 hours
- 2pm–3pm drain = max(0, spa load 3.2 kW + home load 1 kW minus solar_avg_kw)
- 3pm–4pm: spa is off, so net = home load 1 kW minus solar_avg_kw
- If solar > home load in this window, battery actually recharges 3–4pm
- Projected 4pm SOC = 100% minus both drain periods plus any recharge

Outcome	Condition	Action
EXTEND_TO_3PM	Projected 4pm SOC >= 95%	heat_on called — spa continues for another hour
HEAT_OFF_EARLY	Projected 4pm SOC < 95%	heat_off called — spa stops now to protect battery

Even if the spa is extended to 3pm, the `spa_3pm_decision.py` task runs at 3:00pm to decide whether a further extension to 4pm is safe. See section 4.4.

4.4 `spa_3pm_decision.py` — Extend the Spa to 4pm?

`spa_3pm_decision.py`

Runs at 3:00pm daily via Task Scheduler (Solar-Spa3pm)

If the 2pm decision extended the spa to 3pm, this script runs at 3pm to check whether a further hour (3–4pm) is feasible. The criterion is more conservative: we need at least 95% SOC at 5pm (not 4pm), because the 4–5pm solar window is weaker than the 2–3pm window.

The physics calculation

- $\text{solar_avg_kw} = \text{afternoon_kwh (Solcast 2–5pm forecast)} / 3 \text{ hours}$
- $\text{solar at 4–5pm} = \text{solar_avg_kw} \times 0.6$ (sun is declining in winter — about 60% of the 2–5pm average by 4pm)
- $3\text{–}4\text{pm drain} = \max(0, \text{spa load } 3.2 \text{ kW} + \text{home load } 1 \text{ kW} \text{ minus } \text{solar_avg_kw})$
- 4–5pm: spa is off. Net = home load minus declining solar. If solar > home load, battery recharges.
- Projected 5pm SOC must be $\geq 95\%$ to extend

Outcome	Condition	Action
EXTEND_TO_4PM	Projected 5pm SOC $\geq 95\%$	heat_on called — spa continues to 4pm. Solar-SpaOff at 4pm calls heat_off.
HEAT_OFF	Projected 5pm SOC $< 95\%$	heat_off called — spa stops now to protect battery for nightly export.
NO_ACTION	2pm decision was not EXTEND_TO_3PM	Spa was already off — script exits without action.

A Solar-SpaOff task at 4:00pm always calls heat_off regardless. This is the final safety net — the spa will never run into the 6pm export window.

4.5 `spa_control.py` — Direct Spa Control

`spa_control.py`

Called by decision scripts or manually from PowerShell

This is the low-level script that actually talks to the spa controller over your home WiFi. It uses the geckolib library to communicate with the Gecko in.touch 2 module in the spa.

Commands

Command	What it does
python spa_control.py status	Reads and prints current water temp, target, pump states. Saves to spa_status.json.
python spa_control.py heat_on	Raises target temperature to 38.5°C, starting active heating.
python spa_control.py heat_off	Lowers target to 35.0°C — standby mode, no active heating.
python spa_control.py set_temp 37.0	Sets an explicit temperature target.

Spa IP address	[Your Spa IP — replace in SPA_IP constant at top of spa_control.py]
Active heating target	38.5°C (HEAT_SETPOINT in spa_control.py)
Standby target	35.0°C (STANDBY_SETPOINT — maintenance mode, no active heating)
Status file	spa_status.json — updated after every run
Protocol	Gecko in.touch 2 via geckolib (Python asyncio, SelectorEventLoop for Windows)

4.6 Morning Battery Learning Task

morning-battery-learning (SKILL.md)

Runs at ~8:00am daily via Claude scheduled task

This is the most complex task in the system. It runs as a Claude AI task (not a simple Python script) because it needs to connect to the Sigen web app via Chrome, read multiple data points, apply calibration rules, and compose an HTML email.

Step by step

- Step 1: Navigate to the Sigen app (app-aus.sigencloud.com) and extract last night’s data via the browser API
- Step 2: Read the SOC readings at key points — 7am (actually 8am reading), midnight, 10pm, 9pm, 6pm, 2pm, 3pm — plus solar generation, grid import, and actual export kWh
- Step 3: Calculate performance metrics — overnight discharge rate, projection confidence, export delta vs recommendation, solar window actuals
- Step 4: Apply seven learning rules to update the calibration file
- Step 5: Write updated sigen_calibration.json and morning_data.json
- Step 6: Compose and queue the morning summary email

The seven learning rules

Rule	What it updates
Rule 1 — Discharge rate	Rolling 14-day average of actual overnight SOC drop per hour. This is the core model.
Rule 2 — Forecast accuracy	Tracks how often each solar class (HIGH/MEDIUM/LOW/VERY_LOW) matched actual generation. Builds correction factors.
Rule 3 — Target calibration	Adjusts the recommended 9pm SOC target up or down based on whether 7am SOC came in above or below expectations. Uses export-adjusted 7am SOC.
Rule 4 — Solar recharge	Rolling average of actual SOC gain per hour during the 8am–11am solar window, by forecast class.
Rule 5 — Performance summary	Tracks cumulative revenue, export averages, missed opportunity estimates.
Rule 6 — Spa impact	Monitors the SOC drop during the 2–3pm spa window and checks it against the model.
Rule 7 — Free grid recharge	Tracks actual SOC at 2pm to confirm the battery reached 100% during the free grid window.

The projection confidence metric measures how accurately yesterday's 4:30pm SOC projection matched what actually happened overnight. It uses an export-adjusted comparison — if you exported 1 kWh more than recommended, the projection is shifted to match, so your rounding choices don't count as model errors.

4.7 Nightly Battery Recommendation Task

nightly-battery-recommendation (SKILL.md)

Runs at ~4:30pm daily via Claude scheduled task

This task calculates tonight's optimal export strategy and sends you the recommendation before the 6pm export window opens.

How the recommendation is calculated

- Reads actual SOC at 4pm from the Sigen API
- Reads tomorrow's solar forecast (Solcast class + kWh estimates)
- Selects the calibrated target SOC at 9pm for tomorrow's forecast class
- Calculates export kWh = (SOC at 4pm minus drop from 4–6pm minus target at 9pm) times 24 kWh
- Rounds to one decimal place and shows the nearest whole-number options above and below
- Projects the full SOC curve from 4:30pm to 8am next morning (32 data points, every 30 min)
- Saves the projection to last_recommendation in the calibration file for next morning's accuracy check

The formula

Component	Value / source
SOC at 4pm	Read live from Sigen API
Drop rate 4–6pm	<code>load_model.drop_pct_per_hour</code> x 2 hours
Target SOC at 9pm	<code>target_calibration[solar_class].adj_target</code> (calibrated)
Exportable = available SOC	$(\text{SOC 4pm}) - (\text{drop 4–6pm}) - (\text{target 9pm}) \rightarrow \text{convert to kWh}$
Export rate (kW)	$\text{Exportable kWh} \div 3 \text{ hours (6–9pm window)}$

The nightly email includes a battery SOC chart showing the projected curve from 4pm through to 8am next morning, so you can see visually how the recommendation plays out.

5. Data Files — What Each JSON File Stores

All data files live in your Solar scripts folder (e.g. C:\Solar\). These are plain text JSON files you can open in Notepad at any time.

File	Written by	Read by	Contains
sigen_calibration.json	Morning learning task (daily)	All tasks	The system brain — discharge model, solar model, targets, projection history. See Section 6.
solcast_forecast.json	fetch_solcast.py (3pm)	Spa decisions, nightly task	Tomorrow's solar forecast — daily kWh, morning/afternoon kWh, solar class.
morning_data.json	Morning learning task (8am)	10am spa decision	Today's actual 8am SOC (stored as soc_7am), solar class, and total PV kWh.
spa_status.json	spa_control.py (after every run)	Nightly report, manually	Last known spa state — water temp, target temp, pump states, timestamp.
spa_daily_temps.json	10am, 2pm and 3pm spa decision scripts	Nightly report	Today's spa water temp at 10am, 2pm and 3pm, and the decision made at each time.
spa_decision_log.json	10am, 2pm and 3pm spa decision scripts	Manual review	Rolling log of last 60 spa decisions with full detail — reasons, data, outcomes.
pending_email.json	Morning and nightly tasks	send_pending_email.py	Queue file. Nightly and morning tasks write emails here; the 5-min sender picks it up and sends it, then deletes it.

6. The Calibration System — How the System Learns

signen_calibration.json is the most important file. The morning learning task updates it every day. Here is what each section means:

6.1 Discharge Rate Model (load_model)

Measures how fast the battery discharges overnight when it is not connected to solar or the grid (typically midnight to 7am in winter).

Current drop rate	~2.6% SOC per hour (approx 0.63 kW) — example value from a real system
How it's calculated	Average of the last 14 nights of actual readings
Why it matters	Directly determines how much energy you need at 9pm to reach 7am safely
Seasonal tracking	Also stored by calendar month — winter and summer discharge differ

6.2 Solar Forecast Accuracy (forecast_accuracy)

Tracks how well each solar class matched reality over time. Used to apply correction factors to forecasts.

Class	What gets tracked	How it's used
HIGH	Days forecast as HIGH vs days that were actually HIGH	Builds a correction factor — if HIGH is over-predicted, the system adjusts targets
MEDIUM	Same for MEDIUM	Medium days tend to be the least predictable in most locations
LOW	Same for LOW	LOW days that turn out MEDIUM are common — tracked and corrected
VERY LOW	Same for VERY LOW	Small sample early on — fills in over weeks

6.3 Target Calibration (target_calibration)

The recommended 9pm SOC target for each solar class. The system starts with a base target and adjusts the working target (adj_target) up or down by 2% based on actual morning outcomes:

Solar class	Base target (example)	Adjustment logic
HIGH	47% (11.3 kWh)	+2 if avg 7am SOC < target-4; -2 if avg > target+6
MEDIUM	55% (13.2 kWh)	Same rules apply
LOW	62% (14.9 kWh)	Raised when lower-than-expected mornings occur repeatedly
VERY LOW	68% (16.3 kWh)	Rarely adjusted early on (small sample)

The targets look conservative — nearly half the battery. That is correct. The system needs the battery to last from 9pm through to sunrise the following morning (7–10 hours in winter), while also powering your home load overnight.

6.4 Projection Accuracy (last_recommendation.projection_accuracy)

Each morning, the learning task compares yesterday’s projected SOC curve against what actually happened. This score tells you how reliable the model is:

confidence_pct	100% = perfect. Each 1% MAE subtracts 1 point. Above 90% = green, 75–89% = amber, below 75% = red.
overnight_mae_pct	Mean absolute error for the 9pm–8am overnight window only — the cleanest measure of discharge model accuracy.
export_delta_kwh	How much you diverged from the recommended export (positive = you exported more). Tracked so rounding patterns can be studied.
export_window_mae	Error during the 6–9pm window. This naturally differs when export_delta != 0, so it is kept separate.
mean_bias_pct	Positive = battery consistently lasted longer than projected. Negative = consistently shorter.

6.5 Solar Recharge Model (solar_recharge_model)

Tracks how fast the battery charges from solar during the 8am–11am morning window, grouped by forecast class. Example values from a real system:

Class	Avg % SOC gain per hour	Typical kWh gained in 3 hours	Observations
HIGH	~7.4%/hr	~5.3 kWh	grows with use
MEDIUM	~2.4%/hr	~1.7 kWh	grows with use
LOW	~0.7%/hr	~0.5 kWh	grows with use
VERY LOW	~0.3%/hr	~0.2 kWh	grows with use

7. The Email Reports

7.1 Morning Summary Email (~8am)

Sent after the morning learning task runs. Tells you how last night went.

Section	What it shows
Battery at 8am	Current SOC % and kWh vs projected. Colour-coded green/amber/red.
Solar yesterday	Total PV generated and solar class.
Drop rate	Actual overnight drop vs model — updates if there is a significant difference.
Grid import	Whether the battery ran out overnight and you bought grid power.
Projection confidence	How accurate yesterday's 4:30pm projection was (export-adjusted %). Overnight MAE shown.
Export 6–9pm	Recommended vs actual export, delta (your rounding), and revenue earned.
Solar windows	8am–11am and 2pm–5pm actual kWh vs the model's expected average for that solar class.
Spa 2–3pm	Actual SOC drop while the spa was drawing from battery vs expected.
Calibration changes	Any adj_target or drop rate adjustments made this morning.

7.2 Nightly Recommendation Email (~4:30pm)

The main decision support email. Arrives before the 6pm export window opens.

Section	What it shows
Export recommendation	How many kWh to export 6–9pm. Shown with nearest whole-number options.
Export rate	kW rate to set on the Globird/Sigen app (usually 1–4 kW).
Expected revenue	Estimated income from tonight's export at 15c/kWh.
Solar forecast	Tomorrow's Solcast forecast — daily kWh, class, morning and afternoon windows.
SOC projection chart	Animated table showing projected battery % every 30 minutes from 4pm to 8am.
Spa today	Water temp at 10am, 2pm and 3pm, and the decision made at each time.
Reasoning	Full explanation of how the recommendation was calculated.

8. Key Parameters — What Can Be Adjusted

These are the main thresholds in the system. Edit them in the relevant Python file using a text editor, or ask Claude to change them.

Parameter	Current value	File	What to change it for
MIN_KWH_10AM	9.0 kWh	spa_solar_decision.py	Minimum battery at 10am to allow spa heating. Raise for more buffer; lower if spa isn't running enough.
HEAT_BENEFIT	1.0°C	spa_solar_decision.py	Spa only runs if water is at least this far below target. Raise if you want heating only when the spa is notably cold.
HEAT_SETPOINT	38.5°C	spa_control.py	Target temperature when heating is on.
STANDBY_SETPOINT	35.0°C	spa_control.py	Target temperature in standby (maintenance mode).
SPA_LOAD_KW	3.2 kW	spa_afternoon_decision.py, spa_3pm_decision.py	Estimated spa heater draw (3.0 kW element + 0.2 kW pump). Adjust if your spa differs.
HOME_LOAD_KW	1.0 kW	spa_afternoon_decision.py	Background home load estimate used in 2pm and 3pm projections.
TARGET_SOC_4PM	95%	spa_afternoon_decision.py	Minimum battery needed at 4pm to avoid impact on nightly export decision.
TARGET_SOC_5PM	95%	spa_3pm_decision.py	Minimum battery needed at 5pm if spa extends to 4pm.
SOLAR_DECLINE_4_5PM	0.6	spa_3pm_decision.py	Solar at 4–5pm as a fraction of the 2–5pm average. 0.6 = 60% (winter, sun declining). Adjust for your location/season.
BATTERY_KWH	24.0 kWh	spa_afternoon_decision.py	Total battery capacity.
adj_target (HIGH)	~49%	sigen_calibration.json	Calibrated 9pm target for HIGH solar days. Adjusted automatically

Parameter	Current value	File	What to change it for
			— edit manually only to reset.
adj_target (MEDIUM)	~59%	sigen_calibration.json	Calibrated 9pm target for MEDIUM days.
adj_target (LOW)	~70%	sigen_calibration.json	Calibrated 9pm target for LOW days.
drop_pct_per_hour	~2.64%/hr	sigen_calibration.json	Overnight discharge rate. Updated automatically — do not edit manually.

9. Manual Controls

All scripts can be run manually from a PowerShell window. Open PowerShell and type the commands below. Replace C:\Solar\ with your actual path.

9.1 Check Spa Status

```
python C:\Solar\spa_control.py status
```

9.2 Turn Spa On / Off

```
python C:\Solar\spa_control.py heat_on
python C:\Solar\spa_control.py heat_off
```

9.3 Run Spa Decision Manually

Useful for testing or if the scheduled task missed:

```
python C:\Solar\spa_solar_decision.py
python C:\Solar\spa_afternoon_decision.py
python C:\Solar\spa_3pm_decision.py
```

9.4 Fetch Solar Forecast Manually

```
python C:\Solar\fetch_solcast.py
```

9.5 Windows Task Scheduler

To check or manage the scheduled tasks, open Task Scheduler (search in Start menu) and look under the root task folder. The tasks are:

Task name	Schedule	Script
Solar-SendPendingEmail	Every 5 minutes	send_pending_email.py
Solar-FetchSolcast	Daily at 3:00pm	fetch_solcast.py
Solar-SpaDecision	Daily at 10:00am	spa_solar_decision.py
Solar-SpaAfternoon	Daily at 2:00pm	spa_afternoon_decision.py
Solar-Spa3pm	Daily at 3:00pm	spa_3pm_decision.py (decides whether to extend to 4pm)
Solar-SpaOff	Daily at 4:00pm	spa_control.py heat_off (safety net — always runs)
morning-battery-learning	Daily at ~8:00am	Claude scheduled task (SKILL.md)
nightly-battery-recommendation	Daily at ~4:30pm	Claude scheduled task (SKILL.md)

10. Troubleshooting

Symptom	Likely cause	What to do
Spa did not turn on at 10am	Battery below 9 kWh threshold, or water already at target temp	Check <code>spa_decision_log.json</code> for today — the 'reasons' field explains exactly why.
Morning email not received	Morning task failed or email queue not sent	Check <code>pending_email.json</code> — if it exists, the sender task may not be running.
Nightly email arrives with no SOC data	Sigen app was slow or Chrome didn't connect	Run the nightly task manually from Cowork. Check if <code>app-aus.sigencloud.com</code> is accessible.
Spa shows ERROR in decision log	<code>spa_control.py</code> could not connect to spa via <code>geckolib</code>	Run <code>'python spa_control.py status'</code> to test. Check spa is powered on and on WiFi.
Spa heating stopped in eco mode	Spa controller has an economy schedule active	Delete the eco schedule in the Gecko/Balboa app. <code>heat_on</code> sets the target but eco mode overrides it.
Solcast forecast missing	<code>fetch_solcast.py</code> did not run or API error	Run <code>fetch_solcast.py</code> manually. Check <code>solcast_forecast.json</code> — if error key is present, check API key.
Projection confidence below 75%	Model accuracy has dropped — check for anomalous nights	Look at <code>projection_accuracy_history</code> in calibration file. One bad night recovers over 14 days.
<code>adj_target</code> seems too high/low	Calibration learning converged on wrong value (rare)	Edit <code>adj_target</code> in <code>sigen_calibration.json</code> directly, or ask Claude to reset it.

The `spa_decision_log.json` file is the best first stop for any spa-related question. It records the full data and reasoning for every 10am, 2pm and 3pm decision. The calibration file's `last_recommendation` section shows exactly what the nightly task calculated and why.

11. Adapting This System for Your Setup

This guide was written for a specific system (Sigenergy 24 kWh, Globird Zero Hero tariff, Yarrowonga VIC). Here is what you need to change to use it for your own setup:

What to change	Where	Notes
Station ID	sign_calibration.json, JavaScript API calls in SKILL.md files	Your Sigenergy Station ID is in the Sigenergy app under System > Station Info
Spa IP address	spa_control.py (KNOWN_SPA_IP constant near top)	Find it in your router's device list or the Gecko in.touch app
Solcast resource ID	fetch_solcast.py (RESOURCE_ID constant near top)	Create a free Solcast account at solcast.com.au , add your rooftop, copy the resource ID
File paths	All Python scripts (SOLAR_DIR at top of each file)	Change C:\\Solar\\ to your actual path. Must be consistent across all scripts.
Tariff windows	nightly SKILL.md, morning SKILL.md	The export windows (6–9pm premium, 5–11pm base) are hardcoded in the nightly task logic
Battery capacity	BATTERY_KWH in spa scripts, plus calibration targets	Change 24.0 if your battery is a different size. Re-seed calibration adj_targets accordingly.
Spa load	SPA_LOAD_KW in spa_afternoon_decision.py and spa_3pm_decision.py	Measure actual draw with a smart plug or check your spa's spec sheet
Home load	HOME_LOAD_KW in spa_afternoon/3pm decision scripts	Check your grid meter or smart meter app for your typical base load
Timezone	AEST in Python scripts (ZoneInfo), time checks in SKILL.md	Replace Australia/Melbourne with your timezone if different
Email address	Morning and nightly SKILL.md files	Search for the send address in each SKILL.md and update to your own email

Start with just the nightly recommendation task running manually (no scheduler) and compare its suggestions to what you actually do. Once you trust the discharge model (after 2–3 weeks), automate the spa decisions.

12. Glossary

Term	Meaning
SOC	State of Charge — battery level as a percentage (0–100%).
kWh	Kilowatt-hour — a unit of energy. A 24 kWh battery at 50% SOC holds 12 kWh.
kW	Kilowatt — a rate of power. The spa draws 3.2 kW; 3.2 kW for 1 hour = 3.2 kWh.
adj_target	Adjusted target — the calibrated 9pm SOC target after learning. Differs from base_target as the system adapts.
drop_pct_per_hr	How many % SOC the battery loses per hour overnight. Typically 2–3%/hr for a 24 kWh system.
export delta	How many kWh you actually exported minus what the system recommended. Positive = you exported more.
Solcast	Solar forecasting service. Provides estimated generation in kWh for your specific panels.
Solar class	Category for tomorrow's solar generation: HIGH (>25 kWh), MEDIUM (12–25), LOW (4–12), VERY LOW (<4).
Free grid	Globird Zero Hero window 11am–2pm when grid electricity costs nothing. System charges battery from grid + solar during this window.
Export window	6pm–9pm — when you can sell stored energy to the grid for 15c/kWh premium rate.
geckolib	Python library used to communicate with the Gecko in.touch 2 spa controller over WiFi.
SelectorEventLoop	Technical fix for Windows Python — required for the spa scripts to work reliably on Windows 10/11.
MAE	Mean Absolute Error — average deviation between projected and actual SOC values. Lower = better model accuracy.